

SME0827

Simulador de Livro de Ofertas

Relatório técnico do projeto prático

Disciplina SME0827 - Estruturas de Dados Turma: Estatística e Ciência de Dados Professor: Marcos Mansano Furlan	Status do relatório Versão base em desenvolvimento Parte de Felipe já contemplada Demais seções preparadas para atualização
--	--

Integrantes

Nome	Identificação
Eduardo Affonso Boide Santos	Nº USP: 16862544
Fernando Lacerda Dantas	Nº USP: 17097341
Felipe Alirio Baruja	Nº USP: 15636442
Nicolas Gonçalves Follone	Nº USP: 16827586
Victor Hugo Aparecido Galho Trasancos	Nº USP: 16839040

Arquivo de trabalho preparado para receber as atualizações de cada integrante conforme as partes do código forem concluídas.

Índice executivo

Este índice organiza o relatório como um documento vivo: a parte do Felipe já está preenchida e as demais seções foram estruturadas para receber as atualizações dos integrantes conforme o código evoluir.

#	Seção	Status
01	Resumo executivo do projeto	Preenchido
02	Dados da disciplina e composição do grupo	Preenchido
03	Escopo técnico e regras de implementação	Preenchido
04	Arquitetura do arquivo final e integração por pastas	Preenchido
05	Divisão de responsabilidades e status de cada parte	Preenchido
06	Parte 5 - Felipe: menu principal e execução do simulador	Preenchido
07	Contratos de integração entre menu, Order e OrderBook	Preenchido
08	Estruturas de dados previstas	A atualizar
09	Livro de ofertas e motor de match	A atualizar
10	Análise de complexidade teórica	A atualizar
11	Notebook de performance e gráficos	A atualizar
12	Plano de testes	A atualizar
13	GitHub, commits e evidência de participação	Preenchido
14	Roteiro da defesa oral	Preenchido
15	Checklist final da entrega	Preenchido
16	Conclusão provisória e próximos passos	Preenchido

1. Resumo executivo

Este relatório documenta o desenvolvimento do Simulador de Livro de Ofertas, projeto prático da disciplina SME0827 - Estruturas de Dados. O sistema tem como objetivo simular o recebimento, organização e casamento de ordens de compra e venda de ativos, utilizando estruturas lineares implementadas manualmente em Python.

A proposta técnica central é construir, sem estruturas prontas, uma fila de entrada, listas duplamente encadeadas ordenadas, pilha de undo, registros de transação e um motor de match. O arquivo final será único, com nome simulador_livro_ofertas.py, e reunirá as partes individuais desenvolvidas por cada integrante.

Status atual

A parte de Felipe, responsável pelo menu principal, leitura de dados, validações, geração de timestamp e execução do simulador pelo terminal, já está contemplada neste relatório. As demais partes possuem espaços técnicos preparados para atualização conforme forem concluídas.

2. Dados da disciplina e composição do grupo

Item	Descrição
Disciplina	SME0827 - Estruturas de Dados
Turma	Estatística e Ciência de Dados
Professor	Marcos Mansano Furlan
Projeto	Simulador de Livro de Ofertas e Performance de Estruturas
Formato do código	Arquivo Python único ao final da integração
Entregas complementares	Notebook de performance, relatório, GitHub e arquivo compactado final

2.1 Integrantes

Integrante	Nº USP
Eduardo Affonso Boide Santos	16862544
Fernando Lacerda Dantas	17097341
Felipe Alirio Baruja	15636442
Nicolas Gonçalves Follone	16827586

Victor Hugo Aparecido Galho Trasancos

16839040

3. Escopo técnico e regras de implementação

O sistema deve gerenciar ordens de compra e venda e executar negociações quando o preço da melhor compra for maior ou igual ao preço da melhor venda. O desafio do trabalho é implementar as estruturas de dados de forma manual, evidenciando o funcionamento interno de nós, referências e encadeamento.

- Todas as ordens novas entram inicialmente em uma fila de entrada.
- As compras ficam em uma lista duplamente encadeada ordenada por preço decrescente.
- As vendas ficam em uma lista duplamente encadeada ordenada por preço crescente.
- O undo utiliza uma pilha para desfazer a última ordem inserida com sucesso no livro.
- O motor de match compara as melhores ofertas e registra transações quando houver casamento.

Regra de implementação

Não usar list ou collections.deque para implementar fila, pilha ou listas encadeadas. A lógica principal deve ser construída com nós e referências.

4. Arquitetura do arquivo final e integração por pastas

Durante o desenvolvimento, cada integrante produzirá sua parte em uma pasta com o próprio nome. Depois, as partes serão reunidas em um único arquivo final.

```
simulador-livro-ofertas/
|-- README.md
|-- felipe/
|   |-- felipe_menu.py
|-- eduardo/
|   |-- eduardo_classes_lista.py
|-- fernando/
|   |-- fernando_fila_pilha.py
|-- nicolas/
|   |-- nicolas_transacoes.py
|-- victor/
|   |-- victor_livro_match.py
|-- notebook/
|   |-- performance_tests.ipynb
|-- relatorio/
|   |-- relatorio.pdf
|-- final/
|   |-- simulador_livro_ofertas.py
|   |-- link_github.txt
```

No arquivo final, a ordem das partes será: classes base, fila e pilha, transações, livro de ofertas e, por último, menu principal.

5. Divisão de responsabilidades

Integrante	Parte	Status	Commit
Eduardo	Order, Node, DoublyLinkedList	A preencher	feat: implementa classes base e lista duplamente encadeada
Fernando	Queue e Stack	A preencher	feat: implementa fila encadeada e pilha de undo
Nicolas	Transaction	A preencher	docs: adiciona transacoes
Victor	OrderBook e motor de match	A preencher	feat: implementa livro de ofertas e motor de match
Felipe	Menu, main, validações e execução	Pronto - base inicial entregue	feat: adiciona menu principal e execucao do simulador

Cada integrante deverá comentar a própria parte, realizar o commit correspondente e apresentar a seção que implementou. A integração final deverá preservar os nomes de classes e métodos definidos neste relatório.

6. Parte 5 - Felipe: menu principal e execução do simulador

A parte de Felipe já foi construída como interface principal do sistema pelo terminal. Ela é responsável por receber os dados do usuário, validar as entradas, criar ordens e chamar os métodos principais do livro de ofertas.

Item	Descrição
Responsável	Felipe Alirio Baruja - Nº USP: 15636442
Arquivo inicial	felipe/felipe_menu.py
Seção final	Parte 5 - Menu principal e execução do simulador
Commit	feat: adiciona menu principal e execucao do simulador
Status	Pronto como base de integração

6.1 Funções implementadas por Felipe

Item	Descrição
gerar_timestamp()	Gera data e hora da ordem no momento do cadastro.
ler_inteiro()	Valida entradas inteiras, como ID e quantidade.

ler_float()	Valida preço e aceita vírgula ou ponto como separador decimal.
ler_tipo_ordem()	Garante que o tipo da ordem seja C para compra ou V para venda.
ler_ordem()	Lê os dados da ordem e cria um objeto Order.
exibir_cabecalho()	Mostra o cabeçalho do simulador no terminal.
exibir_menu()	Mostra as opções disponíveis para o usuário.
menu()	Controla o loop principal do programa e chama os métodos do OrderBook.
main()	Função principal que inicia a execução do sistema.

6.2 Opções do menu

```

1 - Inserir ordem de compra
2 - Inserir ordem de venda
3 - Processar próxima ordem da fila
4 - Processar todas as ordens da fila
5 - Mostrar livro de compras
6 - Mostrar livro de vendas
7 - Mostrar transações realizadas
8 - Desfazer última ordem inserida no livro
0 - Sair

```

6.3 Como a parte de Felipe se conecta ao restante do sistema

A parte de Felipe cria uma instância de OrderBook e usa esse objeto como ponto central da execução. Por isso, a parte de Victor deve preservar os métodos esperados pelo menu.

```

livro = OrderBook()

livro.add_order(order)
livro.process_next_order()
livro.process_all_orders()
livro.show_buy_orders()
livro.show_sell_orders()
livro.show_transactions()
livro.undo_last_order()

```

Contrato de integração

A lógica interna do OrderBook pode ser construída por Victor, mas os nomes dos métodos acima devem ser mantidos para que o menu funcione sem retrabalho.

6.4 Teste manual da parte de Felipe

```
cd felipe  
python felipe_menu.py
```

Enquanto as demais partes não forem finalizadas, algumas opções podem exibir avisos temporários. Isso é esperado, pois o menu já está preparado para chamar métodos que serão implementados na integração final.

7. Contratos de integração

Para que a união das partes funcione, os nomes abaixo devem ser preservados. Alterações nesses nomes podem quebrar o menu ou a integração entre as estruturas.

```
Order  
Node  
DoublyLinkedList  
Queue  
Stack  
Transaction  
OrderBook  
add_order(order)  
process_next_order()  
process_all_orders()  
show_buy_orders()  
show_sell_orders()  
show_transactions()  
undo_last_order()  
menu()  
main()
```

Também devem ser preservados os tipos de ordem: C para compra e V para venda.

8. Estruturas de dados previstas

8.1 Order, Node e DoublyLinkedList - Eduardo

[A preencher por Eduardo] Esta seção deverá explicar a classe Order, a estrutura Node e o funcionamento da lista duplamente encadeada. Também deverá demonstrar como a lista de compras fica ordenada do maior preço para o menor e como a lista de vendas fica ordenada do menor preço para o maior.

8.2 Queue e Stack - Fernando

[A preencher por Fernando] Esta seção deverá explicar a fila de entrada em comportamento FIFO e a pilha de undo em comportamento LIFO, destacando as operações de extremidade com custo $O(1)$.

8.3 Transaction - Nicolas

[A preencher por Nicolas] Esta seção deverá explicar como uma transação registra o casamento entre uma ordem de compra e uma ordem de venda, incluindo preço de execução, quantidade e timestamp.

9. Livro de ofertas e motor de match

[A preencher por Victor] Esta seção deverá explicar a classe OrderBook, responsável por integrar fila, listas, pilha e transações. O motor de match deverá comparar a melhor compra com a melhor venda e gerar transações quando a condição de casamento for atendida.

Regra de match

Existe negociação quando o melhor preço de compra é maior ou igual ao melhor preço de venda.

Também será necessário documentar o tratamento de casamento total, casamento parcial, atualização de quantidades e remoção de ordens zeradas.

10. Análise de complexidade teórica

Esta seção será consolidada ao final, depois da integração completa. A base teórica prevista é a seguinte:

Operação	Estrutura	Complexidade esperada
Inserção na fila	Queue	$O(1)$
Remoção da fila	Queue	$O(1)$
Push na pilha	Stack	$O(1)$
Pop na pilha	Stack	$O(1)$
Inserção ordenada	DoublyLinkedList	$O(n)$
Busca por ID	DoublyLinkedList	$O(n)$
Remoção de nó conhecido	DoublyLinkedList	$O(1)$
Match	OrderBook	Depende do volume de casamentos e do custo de manutenção do livro

11. Notebook de performance e gráficos

O notebook `performance_tests.ipynb` deverá medir o tempo de processamento do simulador em diferentes volumes de ordens. A proposta inicial é testar 10.000, 50.000 e 100.000 ordens.

- Gerar ordens simuladas de compra e venda.
- Executar o processamento no livro de ofertas.
- Medir tempo total e tempo médio por ordem.
- Gerar gráficos de crescimento do tempo conforme o volume aumenta.
- Interpretar os resultados de acordo com a complexidade das listas encadeadas.

Atualização futura

Esta seção deverá receber os gráficos finais exportados do notebook e a análise empírica escrita pelo grupo.

12. Plano de testes

Os testes finais deverão demonstrar que as principais regras do simulador estão funcionando.

- Inserir ordem de compra sem match.
- Inserir ordem de venda sem match.
- Executar match total entre compra e venda.
- Executar match parcial com sobra de quantidade.
- Mostrar livro de compras ordenado.
- Mostrar livro de vendas ordenado.
- Registrar transações realizadas.
- Executar undo da última ordem inserida no livro.
- Testar fila vazia.
- Testar pilha vazia.

13. GitHub, commits e evidência de participação

O repositório no GitHub será usado para demonstrar a evolução incremental do código e a participação individual. Cada integrante deverá fazer o commit da própria parte.

```
feat: implementa classes base e lista duplamente encadeada
feat: implementa fila encadeada e pilha de undo
docs: adiciona transacoes
feat: implementa livro de ofertas e motor de match
feat: adiciona menu principal e execucao do simulador
fix: integra partes finais do simulador
```

O link do repositório deverá ser inserido no relatório e no arquivo final/link_github.txt.

14. Roteiro da defesa oral

Item	Descrição
Eduardo	Explica Order, Node, DoublyLinkedList, ordenação e inserção $O(n)$.
Fernando	Explica Queue, Stack, FIFO, LIFO e operações $O(1)$.
Nicolas	Explica Transaction e histórico das negociações.
Victor	Explica OrderBook, match total, match parcial e undo.
Felipe	Explica menu, execução, validações e integração com OrderBook.

Cada integrante deverá demonstrar domínio da própria parte e compreender como ela se conecta ao restante do simulador.

15. Checklist final da entrega

- Arquivo final simulador_livro_ofertas.py funcionando.
- Notebook performance_tests.ipynb criado e com gráficos.
- Relatório com análise teórica e empírica.
- README atualizado.
- Link do GitHub no relatório e em link_github.txt.
- Histórico de commits individuais no GitHub.
- ZIP final com todos os arquivos necessários.
- Todos preparados para a defesa oral.

16. Conclusão provisória e próximos passos

Este relatório já estabelece a estrutura completa do documento final e contempla a parte de Felipe, que está pronta como interface de execução do simulador. As próximas atualizações deverão preencher as seções de Eduardo, Fernando, Nicolas e Victor, além de inserir os resultados do notebook de performance e os gráficos finais.

Próximo passo: cada integrante deve entregar seu arquivo .py dentro da pasta com o próprio nome. Depois, as partes serão unidas no arquivo final e o relatório será atualizado com os resultados reais da integração.